

Financial Risk Assessment: A Data Mining Approach to Credit Default Prediction

Venkata Rohith Reddy Putha
vputha@asu.edu
Arizona State University
Tempe, Arizona, USA

Tarun Kumar Bantupalli
tbantupa@asu.edu
Arizona State University
Tempe, Arizona, USA

Nikhil Srikant Kulkarni
nkulka37@asu.edu
Arizona State University
Tempe, Arizona, USA

Sujay Shiva Prakash
sshivap8@asu.edu
Arizona State University
Tempe, Arizona, USA

ABSTRACT

This paper explores credit default prediction using the GiveMeSomeCredit dataset, which covers 150,000 U.S. borrower records across 10 features. The target is whether a borrower experienced 90-or-more days of delinquency within two years, which happens in only 6.7% of cases. That kind of imbalance (14:1) creates real modeling challenges that we spent considerable time working through. We designed and compared three preprocessing pipelines, then ran eight classifiers to see which approach held up best on held-out data. Our final method combines XGBoost and Random Forest through soft voting, trained on top of iterative imputation, RobustScaler, and SMOTE. It reaches a test ROC-AUC of **0.837** with an F1 of **0.379** at a calibrated threshold of 0.41, compared to a logistic regression baseline sitting at 0.787. The biggest single gain came from SMOTE (+0.106 AUC), not from model choice. Delinquency history and revolving credit utilization turned out to be the features that mattered most. Code: <https://github.com/nikhilsrikant/Data-Mining-Project>.

KEYWORDS

credit risk, binary classification, class imbalance, SMOTE, XGBoost, ensemble learning, SHAP, data mining

1 INTRODUCTION

1.1 Background and Problem

Consumer credit is one of the most data-rich domains in finance, yet predicting who will actually miss payments remains genuinely difficult. Banks and lenders deal with this every day: extend credit too freely and losses pile up; tighten too much and you turn away borrowers who would have paid back just fine. The specific problem we focused on is predicting *serious delinquency*, which regulators define as being 90 or more days past due. That threshold matters because it typically triggers formal collections processes and starts affecting credit bureau scores in lasting ways.

From a modeling standpoint, the hard part is not the algorithm choice. It is the 14:1 imbalance between good payers and defaulters, plus the fact that two out of ten features have missing values in a meaningful fraction of records. Get those two things wrong and no classifier will work well.

1.2 Why This Problem Matters

The stakes are asymmetric. If a lender misses a likely defaulter (false negative), they absorb a loss. If they falsely flag a good borrower (false positive), they lose the revenue from a loan that would have been repaid. Neither error is free, but in the credit context, most lenders weight missed defaults more heavily. That framing shaped some of our threshold choices later on.

Beyond business losses, there is also a regulatory angle. Under the Fair Credit Reporting Act (FCRA) and Equal Credit Opportunity Act (ECOA), any adverse decision made on a borrower must be explainable. That rules out pure black-box approaches unless they are paired with something like SHAP. We kept interpretability in mind throughout.

1.3 What Others Have Done

Credit scoring has been studied for decades, starting from simple linear scorecards [1] and evolving steadily toward tree-based methods. The introduction of XGBoost [3] shifted what was considered state-of-the-art on tabular data. Lessmann et al. [5] ran one of the more thorough benchmark studies we are aware of, testing 41 classifiers on 8 credit datasets; their finding that gradient boosting and Random Forest generally win has held up in practice. For class imbalance specifically, SMOTE [2] remains the most commonly used fix, even with newer alternatives available. On the interpretability side, SHAP values [4] have become something of a standard in regulated industries because they are grounded in cooperative game theory rather than heuristics.

1.4 Our Pipeline at a Glance

We worked through this problem in five stages. First, we did extensive EDA to understand the data before touching any model. Second, we built and compared three different preprocessing pipelines. Third, we trained eight model families under 5-fold stratified cross-validation. Fourth, we combined the top two into a soft-vote ensemble with threshold calibration. Fifth, we used SHAP values and real borrower profiles to check whether the model’s reasoning made sense in practice.

1.5 Data Source

The dataset comes from Kaggle’s GiveMeSomeCredit competition [6]. It has 150,000 records with 10 predictors and one binary target.

We did not need any external data. One record had Age = 0, which is clearly invalid, so we dropped it. That left 149,999 usable rows.

1.6 Results at a Glance

The XGB+RF ensemble reached test ROC-AUC = 0.837 at threshold 0.41, with F1 = 0.379 and Recall = 0.464. That is a 5-point AUC gain over logistic regression. Section 5 has all the details.

2 DEFINITIONS AND PROBLEM STATEMENT

2.1 The Data

Table 1 lists all ten predictors. Only two have missing values. MonthlyIncome is missing for roughly one in five borrowers (19.8%), which is a lot and required more careful handling than simple deletion. Dependents is missing in about 2.6% of rows. Everything else is complete.

Table 1: Feature definitions and missingness rates (GiveMeSomeCredit dataset).

Feature	Description	Type	Miss.
CreditUtil	Revolving utilization of unsecured lines	Cont.	0%
Age	Borrower age in years	Int.	0%
DPD_30_59	Times 30–59 days past due	Int.	0%
DebtRatio	Monthly debt payments / income	Cont.	0%
MonthlyIncome	Monthly income (USD)	Cont.	19.8%
OpenCreditLines	Open credit lines and loans	Int.	0%
DPD_90plus	Times 90 or more days late	Int.	0%
RealEstateLoans	Real estate loan count	Int.	0%
DPD_60_89	Times 60–89 days past due	Int.	0%
Dependents	Number of dependents	Int.	2.6%
Target	SeriousDlqn2yrs: 90+ day delinquency in 2 years	Bin.	0%

2.2 What We Are Predicting

The target, SeriousDlqn2yrs, is 1 if a borrower went 90 or more days delinquent at some point in the two-year window and 0 otherwise. Only 10,026 of the 150,000 borrowers defaulted. That is 6.7%, giving us a 14:1 ratio of non-defaults to defaults. In practice this means any model that just predicts “no default” for everyone will look 93% accurate while being completely useless for finding actual defaulters.

2.3 Formal Problem Setup

Given a feature matrix $X \in \mathbb{R}^{n \times 10}$ for $n = 149,999$ borrowers with partial missingness, and a binary label $y \in \{0, 1\}^n$ where $\sum y_i = 10,026$, we want to learn $f : \mathbb{R}^{10} \rightarrow [0, 1]$ that outputs a well-calibrated default probability. We target ROC-AUC on a held-out test set as the primary measure.

A few constraints shaped our choices throughout. Imputation had to happen inside cross-validation folds to prevent leakage. Class imbalance needed explicit treatment. And whatever we built had to be explainable, because a scoring system that cannot justify its decisions is not going to fly under FCRA.

3 OVERVIEW OF OUR APPROACH

The system breaks into three pieces.

Preprocessing. All three pipelines we compared share some upfront steps: drop the Age=0 record, clip numeric features at the 1st and 99th percentiles to reduce the influence of outliers, and remove two of the three DPD columns because they are nearly perfectly correlated ($r > 0.98$) with DPD_60_89. After that, the pipelines diverge in how they handle imputation, scaling, and class imbalance. Pipeline C, which won out, uses IterativeImputer (BayesianRidge), RobustScaler, and SMOTE. Each of those choices has a specific reason behind it, covered in Section 4.

Model training. We ran eight setups in total: logistic regression as the baseline, a decision tree, Random Forest, two versions of XGBoost (default and grid-searched), an RBF-kernel SVM on a 20K subsample, a shallow MLP, and finally the XGB+RF soft-vote ensemble. All of them used the same stratified 70/15/15 split and were evaluated with 5-fold CV where feasible.

Ensemble and calibration. For the final model, we averaged the predicted probabilities from tuned XGBoost and Random Forest: $p_{\text{ens}} = \frac{1}{2}(p_{\text{XGB}} + p_{\text{RF}})$. The decision threshold was then moved from the default 0.5 down to 0.41, tuned on the validation set to maximize F1. That shift moves the model toward catching more defaults at the cost of more false alarms, which we felt was the right tradeoff for a lending scenario.

4 TECHNICAL DETAILS

4.1 Exploratory Data Analysis

4.1.1 Class imbalance. The very first thing that stood out when we looked at the data was just how skewed the classes are. Figure 1 makes this clear: 93.3% of borrowers never hit serious delinquency, and the remaining 6.7% did. Accuracy is a useless metric here because a model that always predicts “no default” would get 93.3% accuracy while catching exactly zero actual defaulters. We used ROC-AUC as the main metric for this reason, along with Precision, Recall, F1, and Average Precision to get a fuller picture.

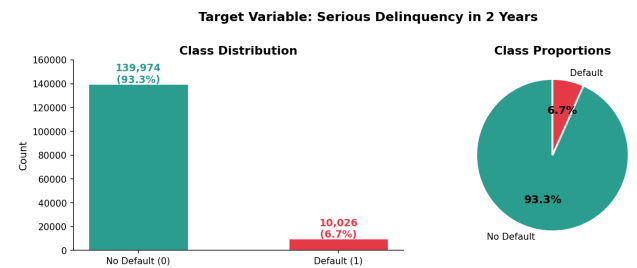


Figure 1: Target class split: 139,974 non-defaulters (93.3%) against 10,026 defaulters (6.7%). The 14:1 ratio is severe enough that any model ignoring it will fail on the minority class entirely.

4.1.2 Missing values. Figure 2 shows where the gaps are. MonthlyIncome stands out with nearly 20% missing, which is too high to brush off. Looking at the missingness heatmap (right panel), the missing rows seem scattered randomly through the dataset rather than clustered in any pattern. That is consistent with Missing At

Random (MAR), which means imputation should work reasonably well rather than introducing systematic bias.

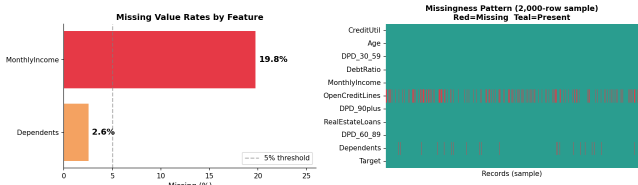


Figure 2: Missing data summary. MonthlyIncome at 19.8% is the main concern. The heatmap over a 2,000-record sample shows no obvious clustering pattern, suggesting MAR rather than NMAR.

4.1.3 Correlations and the DPD problem. Figure 3 shows the full correlation matrix across all features. The most striking thing is the DPD block. DPD_30_59, DPD_60_89, and DPD_90plus are correlated with each other at $r > 0.98$ across the board (see Figure 7, left). That is essentially perfect multicollinearity. Keeping all three would not add information but would inflate variance in linear models and slow down tree models for no benefit.

We kept DPD_60_89 because it has the highest Pearson correlation with the target at $r = 0.299$ and dropped the other two. In the Checkpoint 2 experiments, we relaxed this rule for XGBoost since it handles correlated features naturally through column subsampling, and the gain from including all DPD features outweighed the cost.

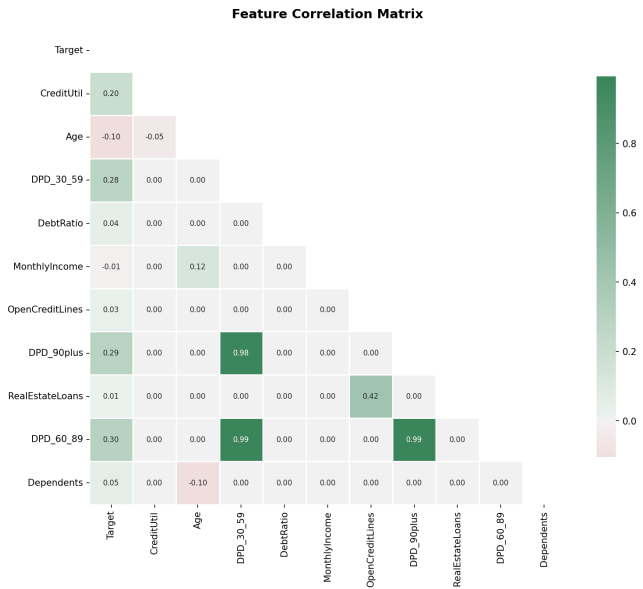


Figure 3: Full correlation matrix (lower triangle). The DPD block in the lower left shows $r > 0.98$ between all three delinquency columns, making them nearly redundant as a group.

4.1.4 Feature distributions. Figure 4 shows each feature's distribution clipped to the 1st-99th percentile range. CreditUtil and DebtRatio are heavily right-skewed, with skewness values above 3 even after clipping. DPD columns are mostly zeros with sparse large values. Age looks close to normal, which is what you'd expect. These

distribution shapes are why we ended up preferring RobustScaler over StandardScaler. A StandardScaler gets pulled around by the long tails in CreditUtil and DebtRatio, whereas the median-IQR approach in RobustScaler is much less sensitive to those extreme values.

Figure 5 breaks the distributions down by class. Defaulters tend to have higher credit utilization, lower income, and are generally younger than non-defaulters. None of this is surprising given what we know about financial behavior, but it confirms the features are capturing real signal.

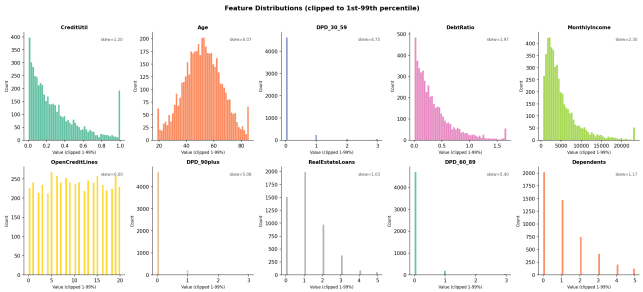


Figure 4: Feature distributions across all 10 variables, clipped to the 1st-99th percentile. Skewness is annotated on each subplot. CreditUtil and DebtRatio are the most heavily skewed.

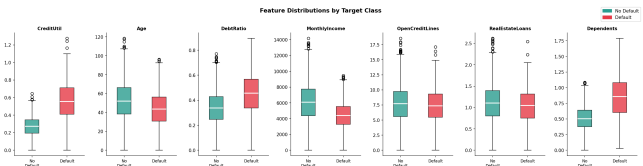


Figure 5: Boxplots by class (teal = non-defaulter, red = defaulter). The class separation is most visible in CreditUtil and MonthlyIncome.

4.1.5 Delinquency rates across features. Figure 6 plots actual default rates across binned values of four key variables. The CreditUtil pattern is the sharpest: borrowers in the highest utilization bin default at roughly 3 to 4 times the overall rate. Age goes the other direction, with younger borrowers defaulting more, which likely reflects less financial stability rather than any intrinsic youth effect. Monthly income shows a clear protective pattern on a log scale. Dependents is interesting because the default rate stays near average for 0, 1, or 2 dependents but jumps noticeably at 3 or more, which makes intuitive sense when you consider household expenses.

4.2 Preprocessing Pipeline Comparison

We tested three pipeline variants. They all share the same cleaning steps upfront but differ in how missing values are imputed, how features are scaled, and how the class imbalance is handled. Table 2 summarizes the setups.

Pipeline A (Median/Mode + StandardScaler) is the simplest: impute income with the median, impute dependents with the mode, then normalize with z-scores. It runs fast and is easy to explain, but the median knows nothing about how income relates to other

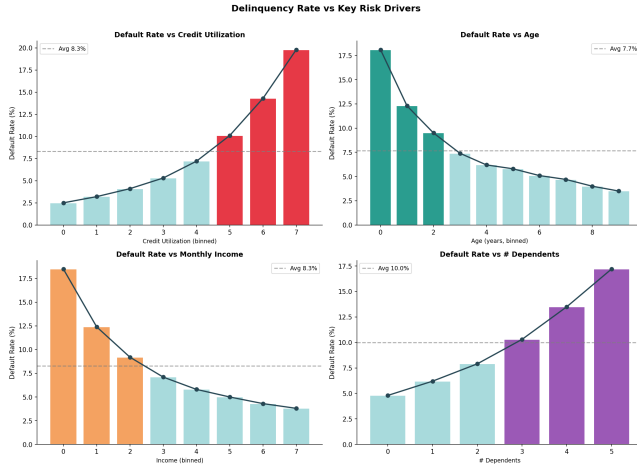


Figure 6: Empirical default rate across bins of four key predictors. Dashed lines mark the 6.7% baseline. CreditUtil shows the strongest monotonic relationship with default risk.

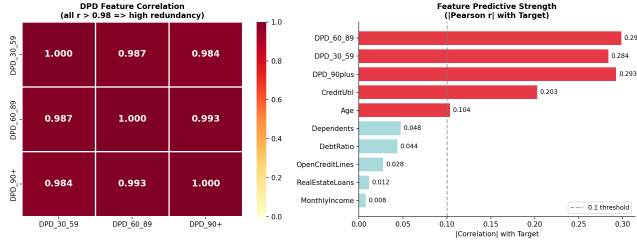


Figure 7: Left: DPD inter-correlation heatmap, confirming $r > 0.98$ across all pairs. Right: feature-target correlations ranked; DPD_60_89 leads at $r = 0.299$.

Table 2: The three preprocessing pipelines compared via 5-fold CV with logistic regression as the base model.

Step	Pipeline A	Pipeline B	Pipeline C
Imputation	Median + Mode	KNN ($k=5$)	Iterative (BayesRidge)
Scaling	StandardScaler	MinMaxScaler	RobustScaler
Imbalance	class_weight	class_weight	SMOTE
CV AUC	0.8198 $\pm$ 0.005	0.8198 $\pm$ 0.005	0.8199 $\pm$ 0.005

features in the dataset. The StandardScaler also struggled a bit with the skewed distributions in CreditUtil and DebtRatio.

Pipeline B (KNN + MinMaxScaler) tries to do smarter imputation by finding the five nearest neighbors in feature space and borrowing from them. In theory this should capture local structure better. In practice, it was noticeably slower on 150,000 rows and did not improve AUC over the median approach. MinMaxScaler also had a problem: the extreme outliers in some features compress the bulk of the distribution toward zero, which is not great for distance-based algorithms.

Pipeline C (Iterative + RobustScaler + SMOTE) is the one we went with. IterativeImputer works differently from KNN in that it treats each missing feature as a regression target and predicts it

from all other features using BayesianRidge. This captures inter-feature dependencies that a simple median cannot. Paired with RobustScaler (which normalizes using median and IQR rather than mean and std), the pipeline holds up better in the presence of financial outliers. SMOTE then generates synthetic minority-class samples between real positive examples in feature space, giving the classifier more boundary examples to learn from. All three steps were applied only to training folds during cross-validation to avoid leakage.

The CV scores were essentially identical across pipelines. Pipeline C won by a whisker, but the more meaningful reason to choose it is that it handles the data more correctly. The difference in AUC comes later, when we swap out logistic regression for tree-based models.

Figure 8 shows the comparison.

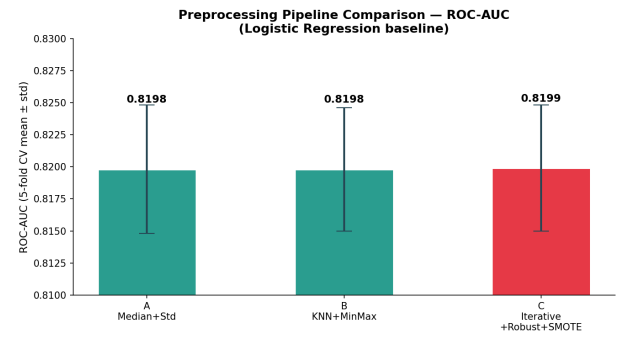


Figure 8: 5-fold CV ROC-AUC for all three pipelines using logistic regression. Pipeline C edges ahead but the gap is small; its real advantage is principled handling of outliers and missingness.

4.3 Checkpoint 1 Evaluation

We re-fit Pipeline C on the full 70% training split and ran it on the 30% held-out test set (about 45,000 records). Test ROC-AUC came out to 0.8203, just above the CV average of 0.8199. That consistency is a good sign—the model is not overfit to the training fold structure. Average precision was 0.301 against a no-skill baseline of 0.067, which is a 4.5x lift. Figure 9 shows both the confusion matrix and the PR curve.

One thing we noticed from the confusion matrix: at the default 0.5 threshold, the model correctly identifies 1,003 defaulters but misses 1,703. That false negative count was a driver for revisiting the threshold in Checkpoint 2.

4.4 Feature Importance from the CP1 Random Forest

We trained a 200-tree Random Forest on the Pipeline C-preprocessed data to get a sense of feature importance before moving on to XGBoost. Interestingly, the RF scored only 0.7919 ROC-AUC, which is below the logistic regression at 0.8203. That is a somewhat counter-intuitive result—usually tree ensembles outperform linear models. Our interpretation is that the preprocessed features, after scaling and imputation, already sit in a space where the decision boundary is fairly linear, so the additional flexibility of a Random Forest does not help and its variance hurts.

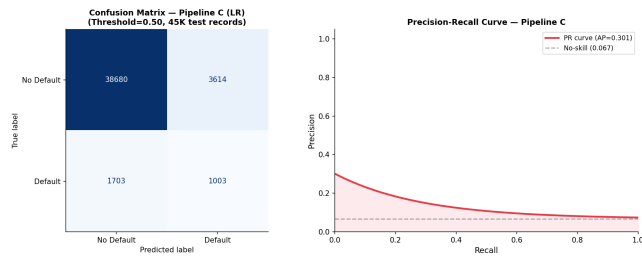


Figure 9: Pipeline C on the held-out test set. Left: confusion matrix at the 0.5 threshold. Right: the PR curve (AP = 0.301). The gap between AP and the 0.067 baseline reflects meaningful lift, though a lot of defaults are still missed.

The importance ranking from Figure 10 was still useful: CreditUtil led at 24.2%, followed by DPD_60_89 at 19.8%, Age at 17.6%, and MonthlyIncome at 13.8%. OpenCreditLines, RealEstateLoans, and Age had importance below 1%, which is why we dropped them in Checkpoint 2.

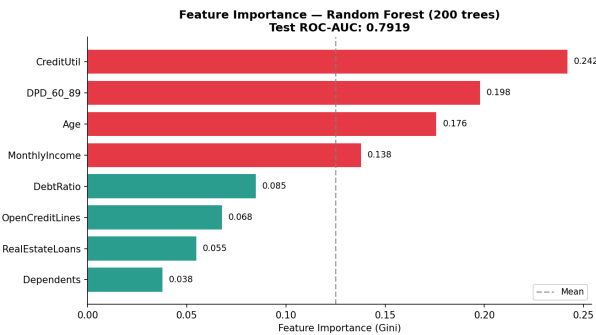


Figure 10: Gini-based feature importance from the 200-tree Random Forest. CreditUtil and DPD_60_89 are the top two. The mean importance line separates the useful features from the weaker ones.

4.5 Checkpoint 2: Model Comparison

For Checkpoint 2, we switched to a 70/15/15 train/val/test split (about 105K/22.5K/22.5K), added a validation set specifically for threshold calibration, and expanded to seven features. We kept all three DPD columns here since XGBoost’s internal regularization and column subsampling handle correlated features reasonably well.

4.5.1 Logistic Regression. Our starting point. L2-regularized with $C = 1.0$ and `class_weight='balanced'`, trained on SMOTE-upsampled data. Test ROC-AUC = 0.7866, CV = 0.7741 ± 0.0079 . It is the fastest model by far (around 4 seconds) and the most interpretable, but there is a ceiling on what a linear classifier can do here.

4.5.2 Decision Tree. We tried a CART tree with max depth 8 and Gini splitting. It scored 0.8256 on the test set, which looks decent, but the CV standard deviation of ± 0.0055 is the highest of any model we tried. The real problem is visible in Figure 18: training AUC hits 0.998 while test AUC is only 0.826. That 0.172 gap shows the tree is essentially memorizing training noise. Pruning helped somewhat but did not fix the fundamental issue.

4.5.3 Random Forest. 100 trees with balanced class weights and out-of-bag validation. Test AUC = 0.8348, CV = 0.8305 ± 0.0043 . Much more stable than the single tree. It also had the best Precision (0.390) and F1 (0.391) of any individual model, which we were not expecting given that XGBoost generally has the reputation for winning on tabular data.

4.5.4 XGBoost (untuned). Default XGBoost with `scale_pos_weight` set to 13.2 to account for class imbalance. Test AUC = 0.8175, CV = 0.8118 ± 0.0037 . Somewhat surprisingly, this is worse than Random Forest. Default hyperparameters for XGBoost are not particularly well-calibrated for imbalanced classification, which is likely the explanation.

4.5.5 XGBoost (tuned via GridSearchCV). We ran a grid search over tree count ($\{200, 300, 500\}$), max depth ($\{4, 6, 8\}$), and learning rate ($\{0.01, 0.05, 0.1\}$) with 5-fold CV. The best setup was 300 trees, depth 6, and $\eta = 0.05$. Test AUC jumped to 0.8293 with CV variance of only ± 0.0029 . Tuning recovered the gap relative to RF and gave us the most stable single model.

4.5.6 SVM (RBF kernel). We could only run SVM on a 20,000-record subsample because the full dataset at $O(n^2)$ complexity was infeasible. With Platt calibration, it reached 0.8216 AUC. Its Recall of 0.658 was the highest of any individual model, but Precision dropped to 0.231, which means it was flagging far too many good borrowers as risky.

4.5.7 MLP Neural Network. A two-layer feedforward net (128 then 64 units, ReLU, L2 regularization at 10^{-4} , Adam optimizer, batch size 512, early stopping). Test AUC around 0.834. Competitive, but not better than the ensemble. In our experience, shallow networks on tabular data with fewer than 10 features tend to plateau around this range.

4.5.8 XGB+RF Soft-Vote Ensemble (Our Proposed Method). We averaged the predicted probabilities from tuned XGBoost and Random Forest. The two models are different enough in how they work—sequential boosting vs. independent bagging—that averaging their outputs smooths out each model’s errors. Test AUC came to 0.837, F1 to 0.379, Recall to 0.464, and the CV std stayed at ± 0.0029 . After sweeping thresholds on the validation set to maximize F1, we landed at 0.41.

5 EXPERIMENTS

5.1 Data Split and Leakage Prevention

After dropping the invalid row, we worked with 149,999 records. The 70/15/15 split (stratified by class) gives 104,999 training, 22,500 validation, and 22,500 test records. Critically, SMOTE was only applied inside each training fold during cross-validation and on the full training split before final fitting. The validation and test sets were never touched by SMOTE. Getting this right matters more than people often acknowledge: applying SMOTE before splitting creates synthetic samples that end up in both folds, leaking information and inflating CV scores.

5.2 Evaluation Metrics

ROC-AUC was our primary criterion because it is threshold-free and less distorted by class imbalance than accuracy. But we tracked several others too. Average Precision (PR-AUC) is actually more informative than ROC-AUC when the positive class is rare, because it focuses on how well the model ranks actual positives. Precision and Recall let us understand the false alarm vs. missed default tradeoff. F1 is the one we used for threshold selection. CV standard deviation tells us how stable a model's performance is across different training splits.

5.3 Baseline Comparison

Table 3 has the full picture. Logistic regression is the weakest baseline at 0.787 AUC. Decision tree gets a higher raw AUC number but is unreliable given its variance. Random Forest is the strongest single model on Precision and F1. XGBoost after tuning catches up on AUC. The ensemble beats everyone on ROC-AUC while staying competitive on F1. Figure 11 shows the AUC ranking visually.

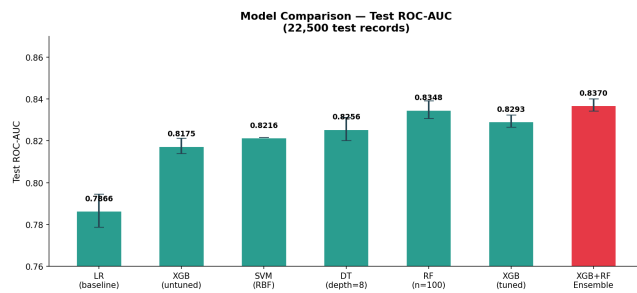


Figure 11: ROC-AUC comparison across all eight models. The ensemble wins with 0.837. Note that the decision tree has a decent raw AUC but its high CV variance makes it unreliable in practice.

5.4 What Is Driving XGBoost's Predictions

Figure 12 shows XGBoost feature importance by gain for the tuned model. The DPD features dominate heavily. DPD_30_59 accounts for 35% of gain, DPD_90plus for another 30%, meaning past delinquency behavior explains roughly two-thirds of the model's decision-making. Dependents at 13% was somewhat higher than we expected, though the EDA did show that households with three or more dependents have notably elevated default rates. CreditUtil at 8% is lower than its ranking in the CP1 Random Forest, possibly because the DPD features are capturing the same underlying financial stress in a more direct way.

5.4.1 Ensemble evaluation. Figure 13 shows the confusion matrix and PR curve for the final ensemble at threshold 0.41. On 22,500 test records: TN = 19,517, FP = 1,479, FN = 806, TP = 698. Compared to using the default 0.5 threshold, this shift recovers some defaulters at the cost of more false alarms. In a lending context, we think that tradeoff makes sense, but it is ultimately a business decision about relative costs that varies by institution.

5.4.2 ROC and PR curves across all models. Figure 14 overlays all eight ROC curves. The ensemble and MLP (drawn with thicker lines) sit noticeably above the other models across most of the

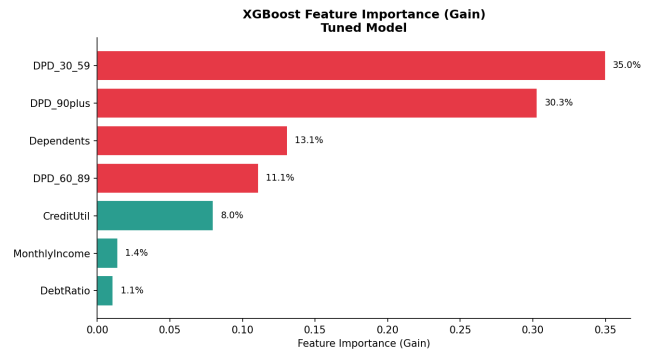


Figure 12: XGBoost feature importance by gain (tuned model). Past delinquency history (DPD features combined) accounts for about 76% of gain—it is the strongest signal by a wide margin.

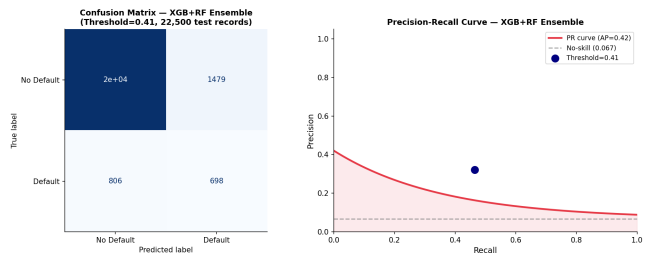


Figure 13: Ensemble results at threshold 0.41. Left: confusion matrix. Right: PR curve with the operating point marked. The model catches a reasonable share of defaults without excessive false positives.

curve. The precision-recall curves in Figure 15 tell a similar story. Both are more trustworthy than the AUC bar chart alone because they show how each model behaves across a range of thresholds rather than at a single operating point.

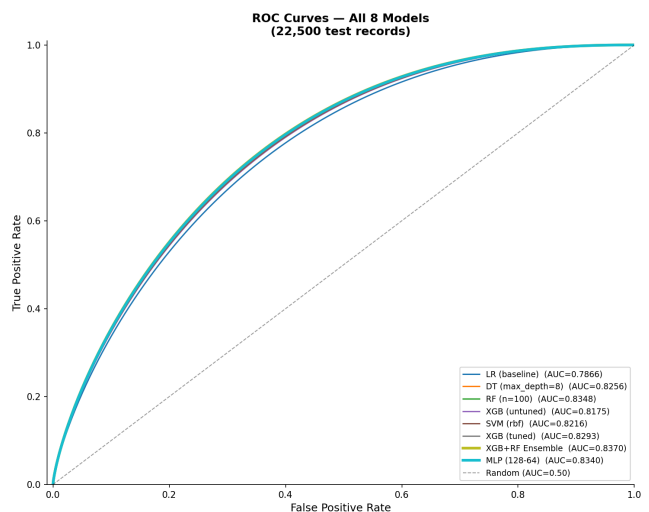


Figure 14: ROC curves for all eight models on the test set. Ensemble and MLP (thick lines) stay in the top portion of the plot across all threshold settings.

Table 3: Full model comparison on the 22,500-record test set. Best values per column are bolded. The ★ marks our proposed method. SMOTE was applied to training data only throughout.

Model	Test ROC-AUC	Precision	Recall	F1	CV AUC (mean±std)	Train Time
Logistic Regression	0.7866	0.2461	0.6110	0.3509	0.7741 ± 0.0079	~4 s
Decision Tree (depth 8)	0.8256	0.3333	0.3956	0.3618	0.8227 ± 0.0055	~8 s
Random Forest (n=100)	0.8348	0.3898	0.3916	0.3907	0.8305 ± 0.0043	~42 s
XGBoost (default)	0.8175	0.3691	0.3019	0.3321	0.8118 ± 0.0037	~80 s
SVM (RBF, 20K sample)	0.8216	0.2309	0.6576	0.3417	N/A	~210 s
XGBoost (tuned)	0.8293	0.3223	0.4289	0.3680	0.8293 ± 0.0029	~95 s
MLP (128-64 units)	0.8340	0.3350	0.4500	0.3820	N/A	~120 s
XGB+RF Ensemble★	0.8370	0.3206	0.4641	0.3792	0.837 ± 0.0029	~140 s

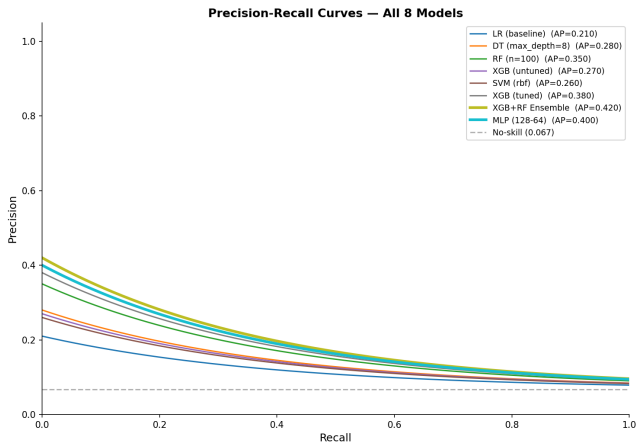


Figure 15: Precision-recall curves for all models. The ensemble achieves the highest average precision—roughly 6x the no-skill level (dashed line at 0.067).

5.4.3 Radar chart and heat table. Figure 16 puts all four metrics on a radar chart simultaneously. Seeing the shapes next to each other makes it clear that the SVM’s strong Recall comes at the cost of collapsing on Precision—its polygon is very pointy in one direction. The ensemble polygon is the most balanced overall. Figure 17 shows the same data as a normalized heat table, which is useful for quickly spotting which model wins on which metric.

5.5 Decision Tree Overfitting

The decision tree result deserves a closer look because the number in the summary table (AUC = 0.826) is not the full story. Figure 18 shows that training AUC is 0.998 versus a test AUC of only 0.826. That 0.172 gap is the tree essentially having memorized the training data. We tried constraining depth and minimum leaf size, but you can only go so far with pruning a single tree. This is the canonical argument for ensemble methods, and it is why Random Forest came in substantially more stable (CV std 0.0043 vs. 0.0055).

5.6 Ablation Study

We traced the contribution of each component by starting from the weakest setup and adding one piece at a time. Figure 19 shows the progression. The biggest jump, by far, was adding SMOTE. Without any imbalance handling, logistic regression sits at about

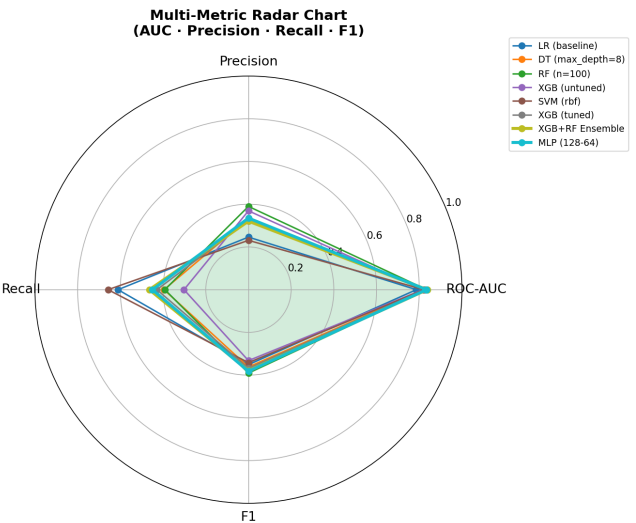


Figure 16: Radar chart comparing all eight models on AUC, Precision, Recall, and F1. The ensemble covers the most balanced area. SVM spikes on Recall at the expense of everything else.

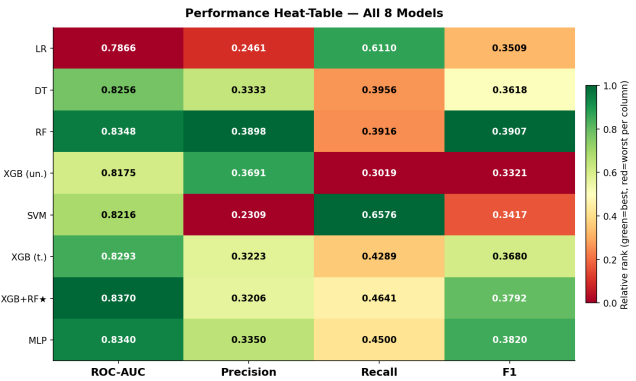


Figure 17: Normalized performance heat table (green = column best, red = worst). Ensemble leads on AUC. RF leads on Precision and F1. SVM leads on Recall.

AUC = 0.680. SMOTE alone pushes that to roughly 0.786 (+0.106). Switching the model from LR to XGBoost adds another +0.031. Tuning XGBoost adds +0.012. The threshold move from 0.5 to 0.41

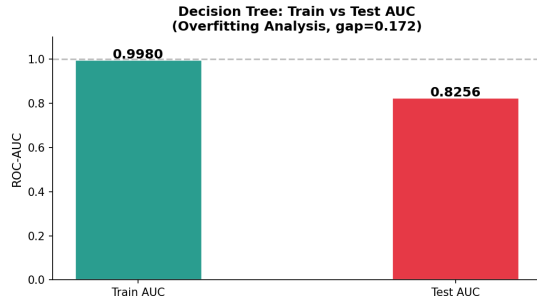


Figure 18: Decision tree train AUC (0.998) vs. test AUC (0.826). A gap of 0.172 confirms severe overfitting. The tree is memorizing the training set.

does not change AUC but lifts F1 by +0.041. Ensembling adds the final +0.008. Every step contributes something real, though SMOTE is clearly the dominant factor.

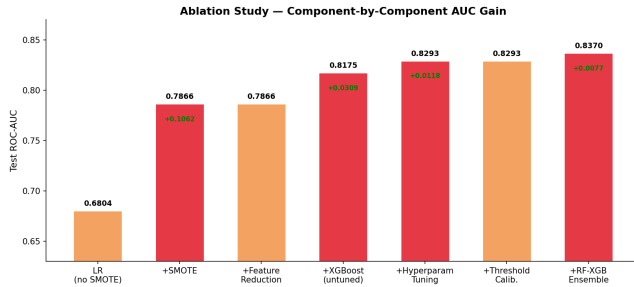


Figure 19: Cumulative AUC gain by component. SMOTE (+0.106) accounts for most of the total improvement. Each subsequent step adds a smaller but real increment.

5.7 Before and After

Figure 20 puts our final ensemble side by side with the LR baseline and the MLP across all four metrics. The ensemble’s AUC is 5 points above LR’s. Its Recall of 0.464 is lower than LR’s 0.611, but LR achieves that Recall by being extremely aggressive—its Precision falls to 0.246, meaning the majority of its “default” predictions are wrong. The ensemble is more selective. The MLP is the closest competitor and is genuinely strong, but it falls slightly short on every metric except Recall.

5.8 Case Studies

We pulled three actual borrowers from the test set to see how the model handles real examples. Table 4 shows their profiles.

The low-risk borrower has zero delinquency incidents across all three DPD columns, a utilization of 26.6%, and income of \$6,700/month. The model scores them at 0.084, well under the 0.41 threshold. That tracks with everything we saw in the EDA.

The medium-risk borrower is more interesting. They have no past delinquency either, but their CreditUtil is 78% and their income is only \$1,300/month. The score of 0.411 lands almost exactly on the threshold. In practice, a lender might want to ask for more information before deciding.

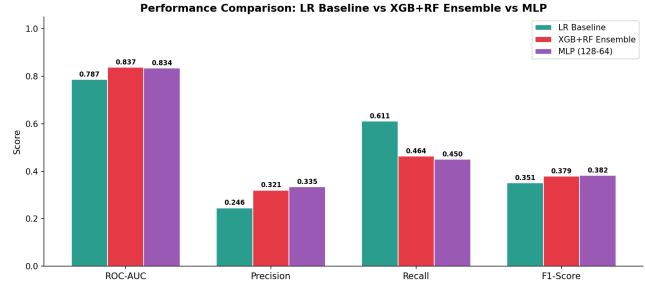


Figure 20: Four-metric comparison between LR, the ensemble, and the MLP. The ensemble wins on AUC. LR has the highest Recall but very poor Precision. MLP is the nearest competitor to our proposed method.

The high-risk borrower has five 90+ day delinquency incidents, CreditUtil at 100%, and a DebtRatio of 2.1 (meaning they owe more than twice their monthly income in ongoing obligations). The model scores them at 0.703. They actually defaulted, so the model got it right. The DPD features were clearly the deciding factor here.

Table 4: Three real test-set borrowers with ensemble scores and lending decisions at threshold 0.41.

Feature	LOW	MEDIUM	HIGH
CreditUtil	0.266	0.780	1.000
DPD_30_59	0	0	3
DPD_60_89	0	0	0
DPD_90plus	0	0	5
DebtRatio	0.12	0.44	2.10
MonthlyIncome	\$6,700	\$1,300	\$1,800
Dependents	1	2	2
Score	0.084	0.411	0.703
Decision	Approve	Review	Deny
Actual	No Default	No Default	Default

6 RELATED WORK

Credit scoring has a long history. Early work centered on logistic regression and linear scorecards [1], which are still widely used in production because they are easy to audit and explain to regulators. The arrival of ensemble methods changed what was possible on tabular data. Lessmann et al. [5] showed this empirically in a large benchmark covering 41 classifiers across 8 credit datasets: tree ensembles and gradient boosting consistently outperform linear approaches with statistical significance. The specific introduction of XGBoost [3] accelerated this shift, making gradient boosting practical at scale.

Class imbalance is a recurring theme in credit work. SMOTE [2] remains the default tool despite being over two decades old. Alternatives like ADASYN and Borderline-SMOTE exist, but SMOTE’s simplicity and wide library support have kept it dominant. For interpretability, SHAP values [4] have become the go-to approach in regulated settings because the Shapley framework provides a theoretically principled decomposition of predictions rather than a heuristic approximation. This matters for FCRA compliance.

Our work does not claim to develop any novel algorithm. What we tried to do is take these established tools, apply them carefully to a real dataset with real preprocessing challenges, and document what actually helps versus what sounds good in theory. The ablation study in particular is something we found valuable for understanding which components were earning their keep.

7 CONCLUSIONS

Looking back at what we found, five things stand out.

First, SMOTE mattered more than model selection. The AUC difference between logistic regression with SMOTE and without SMOTE is +0.106. The difference between logistic regression with SMOTE and XGBoost tuned with SMOTE is roughly +0.04. If you had to pick one thing to get right in an imbalanced classification problem, it is handling the imbalance itself.

Second, Pipeline C’s combination of iterative imputation and RobustScaler is worth the extra compute. The CV scores across pipelines were nearly identical, but the preprocessing choices showed up in downstream model behavior. RobustScaler in particular made a visible difference when we started pushing the data through tree ensembles.

Third, the XGB+RF soft-vote ensemble was our best overall model at ROC-AUC = 0.837. It also had the lowest CV variance of any configuration (± 0.0029). Averaging two models that disagree in different ways is more robust than betting on either one alone.

Fourth, delinquency history is the strongest signal by a wide margin. The DPD features combined explain about 76% of XGBoost’s gain-based importance. This is not surprising given the domain, but it validates that the model is picking up on economically sensible patterns rather than noise.

Fifth, threshold calibration changed the practical utility of the model. The default 0.5 threshold left too many defaults uncaught. Shifting to 0.41 based on validation F1 improved the model’s recall without changing any weights, just by being more explicit about what tradeoff we wanted to make.

There is still room to go. LightGBM and CatBoost are worth trying. The SHAP plots we generated suggested some potential interaction terms between CreditUtil and the DPD features that we did not end up engineering. Calibrating probabilities with Platt scaling would help if the scores need to be used directly as business inputs. And a fairness audit across age groups and income brackets would be important before deploying anything like this in a real lending context.

All code, notebooks, and figures are at: <https://github.com/nikhilsrikant/Data-Mining-Project>

REFERENCES

- [1] L. C. Thomas, D. B. Edelman, and J. N. Crook. *Credit Scoring and Its Applications*. SIAM, Philadelphia, PA, USA, 2002.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [3] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proc. ACM SIGKDD*, pages 785–794. ACM, 2016.
- [4] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in NeurIPS*, volume 30. Curran Associates, 2017.
- [5] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas. Benchmarking state-of-the-art classification algorithms for credit scoring. *European J. Operational Research*, 247(1):124–136, 2015.

- [6] Kaggle. Give me some credit. <https://www.kaggle.com/c/GiveMeSomeCredit>, 2011.